Day 160: Week 23 Build: Neural Collaborative Filtering from the Week's Parts

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 7 | Complete study guide

Watch: Neural Collaborative Filtering (NCF) for recommendation systems explained (Data Science in your pocket) | Neural Collaborative Filtering (NCF) Explanation & Implementation in Pytorch (Steeve Huang)

Script: `day160_neural_collaborative_filtering.py` · every number in this guide is copied by program from `day160_run_output.txt`, the real output of that script.

Neural collaborative filtering (He and others, 2017) learns from watched / not watched data with a small network: GMF (a weighted element-wise product), an MLP over the two embeddings, and NeuMF (both side by side). All three are built from scratch in NumPy with hand-written backpropagation and Adam, and judged with everything the week built. None beat Day 158's implicit ALS on Day 154's test: the best, NeuMF, scored precision@10 0.3475 against 0.3672, a graded-NDCG gap of -0.0275 whose 95% interval excludes 0. Fine-tuning NeuMF never improved on its starting point (a trained GMF and MLP averaged). With the paper's sampled protocol NeuMF would have looked best (0.6649 against 0.6638); ranked against every film, MLP led, and every one of those gaps was smaller than the noise of a single hit rate (0.0116).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

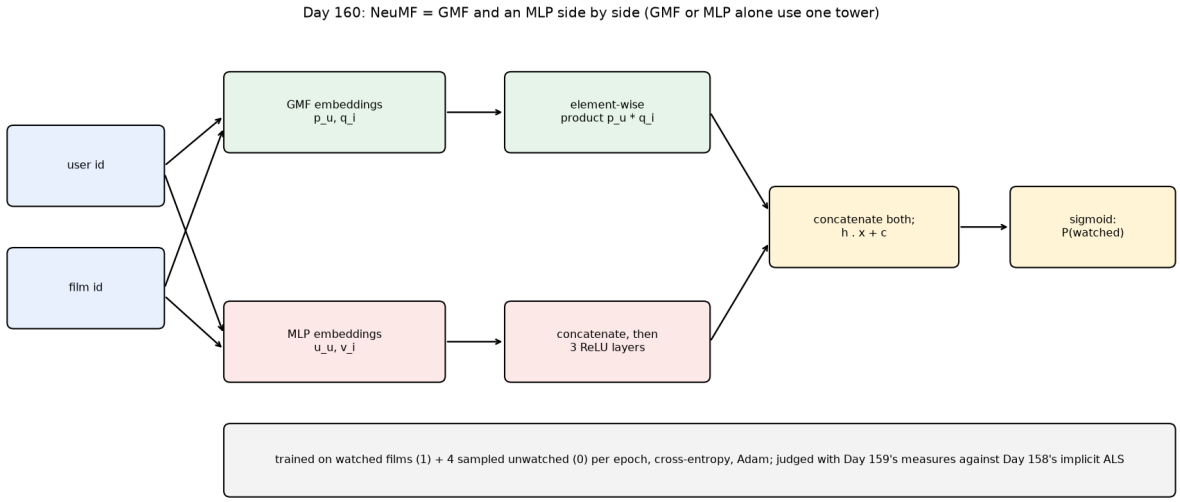
Matrix factorisation scores a user and a film by the dot product of their embeddings. **Neural collaborative filtering (NCF)** lets a network learn that scoring function instead. **GMF** multiplies the two embeddings number by number and learns a weight for each product; with all weights equal it is the dot product. The **MLP** puts the two embeddings side by side and passes them through ReLU layers. **NeuMF** runs both and joins them at the output.

The network is trained as a classifier: films the user watched are 1, films sampled at random from those they did not watch are 0 (4 per watched film, redrawn every epoch), and the loss is binary cross-entropy.

REAL-WORLD SCENARIO

A streaming service already has a matrix-factorisation recommender. A team proposes a neural one, citing a paper where it won. Before replacing anything they need a fair test: the same data, the same tuning rules, the same measures, and the strongest existing model as the bar to beat.

SYSTEM DIAGRAM



NeuMF: two embedding pairs, a GMF tower and an MLP tower, joined at one output and a sigmoid.

Built from the week's parts, imported unchanged: Day 154's MovieLens data and 80/20 split, Day 155's PureSVD, Day 156's genre flags, Day 157's weighted blend, Day 158's implicit ALS and co-occurrence, and Day 159's measures, bootstrap, leave-last-out split and sampled evaluation. Today's build has no exercise file: it is the exercise files, imported. Every setting is chosen on Day 154's validation part of the training split; the test set is used once, in section 4.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The network, and a gradient check

DEFINITION — start here, no prior knowledge assumed

Each user has two embeddings (one for GMF, one for the MLP), and so does each film. GMF's part is $p_u * q_i$ (element-wise product). The MLP's part joins u_u and v_i and passes them through three ReLU layers of widths 2d, d and d/2. The output is a weighted sum of both parts plus a bias, and a sigmoid turns it into a probability that the user watched the film.

Backpropagation is written by hand. A user or film can appear more than once in a batch, so its gradient rows are added with np.add.at rather than assigned. The whole pipeline, forward pass through loss with the penalty on, is checked against finite differences.

REAL-WORLD SCENARIO

A judge scores a pairing two ways: one checks whether each trait matches, one forms an overall impression, and a final weighting combines the two.

MATHEMATICS

$$GMF: x = p_u * q_i; \quad MLP: a_0 = [u_u, v_i], a_{l+1} = max(0, a_l W_l + b_l); \quad z = h \cdot [x, a_3] + c; \quad P(watched) = 1 / (1 + e^{-z})$$

$$loss = -mean[y \log p + (1 - y) \log(1 - p)] + (lam / 2B) \Sigma_{\text{embedding rows in the batch}} |row|^2$$

IMPLEMENTATION

```
def ncf_forward(F, u, i, mode):
    d = F["P"].shape[1]; C = {}
    g = F["P"][u] * F["Q"][i] if mode != "mlp" else np.zeros((len(u), d))
    if mode != "gmf":
        a = np.concatenate([F["U"][u], F["V"][i]], 1); C["acts"] = [a]
        for l in range(3):
            a = np.maximum(a @ F[f"W{l}"] + F[f"b{l}"], 0.0); C["acts"].append(a)
    else:
        a = np.zeros((len(u), F["h"].shape[0] - d))
    C["x"] = np.concatenate([g, a], 1)
    return C["x"] @ F["h"] + F["c"][0], C
```

ACTUAL OUTPUT

```
    gmf: 201 weights; largest gap between backprop and finite differences 8.8e-11
    mlp: 201 weights; largest gap between backprop and finite differences 1.5e-10
  neumf: 201 weights; largest gap between backprop and finite differences 8.6e-11
real size (GMF 64, MLP 32): 258849 weights; one epoch = 5 x the watched films, in batches of 256
Actual output from this lesson's run.
```

What it means

Backprop and finite differences agree to within 1.5e-10 for all three variants, so the gradients are right before any training starts. At the real size there are 258849 weights, most of them embeddings.

WHY CHOOSE THIS?

A network can learn interactions a dot product cannot express; GMF with learned weights contains matrix factorisation as a special case.

WHY NOT THIS? (TRADEOFFS)

Many more settings (sizes, penalties, learning rate, epochs, negatives) and slow training; nothing guarantees the extra freedom helps.

2. A first-draft bug: an L2 penalty inside Adam

DEFINITION — start here, no prior knowledge assumed

The first draft added lam × weight to every gradient at every step, the usual L2 penalty, with lam = 10^{-5}. GMF did not learn. Three versions were compared for 8 epochs on GMF with 16 numbers per embedding: the first draft (decay on every weight), decay on every embedding row only, and the fix: a penalty only on the embedding rows the batch uses, lam × row / B.

REAL-WORLD SCENARIO

A tax charged on every account every day, used or not, drains the dormant accounts even when it is tiny.

IMPLEMENTATION

```
    for k, idx in emb:
        np.add.at(G[k], idx, lam * F[k][idx] / B)
```

ACTUAL OUTPUT

```
  first draft: decay 1e-5 on every weight: validation precision@10 by epoch 0.027, 0.027, 0.027, 0.027, 0.027, 0.027, 0.027,
      0.027
                                          largest |film embedding| 0.0000, largest |output weight| 0.0000
        decay 1e-5 on every embedding row: validation precision@10 by epoch 0.069, 0.075, 0.073, 0.073, 0.069, 0.074, 0.073,
            0.072
                                          largest |film embedding| 1.3124, largest |output weight| 4.1327
    penalty 0.01 on the batch's rows only: validation precision@10 by epoch 0.011, 0.008, 0.010, 0.018, 0.073, 0.078, 0.087,
        0.088
                                          largest |film embedding| 1.4636, largest |output weight| 5.0893
Actual output from this lesson's run.
```

What it means

The first draft stayed flat at 0.027 and every film embedding and output weight went to 0.0000. A likely reason, not tested here: GMF's score is a product of three small numbers (p, q and h), so its gradients start tiny; Adam divides each step by the size of recent gradients, so a decay term, however small its lam, gets full-sized steps while the data gradient is near zero. Decay on the embeddings only did not collapse but stalled near 0.069–0.075. The penalty on the batch's rows learned slowly at first and reached 0.088 by epoch 8.

In a trial run outside this script, per-batch penalties of 0.1 and above also collapsed GMF, so GMF's grid stops at 0.03. Decoupled weight decay (AdamW) is the standard alternative; it was not tried here.

WHY CHOOSE THIS?

The per-batch penalty touches only rows the batch used, so rare films are not shrunk while they are absent.

WHY NOT THIS? (TRADEOFFS)

Its strength now depends on how often a row appears, and it interacts with Adam in ways that must be checked, not assumed.

3. Tuning on validation, and NeuMF three ways

DEFINITION — start here, no prior knowledge assumed

GMF: sizes 32, 64, 128 × penalties 0.003, 0.01, 0.03. MLP: sizes 32, 64, 128 × penalties 0.03, 0.1, 1. Each runs 20 epochs; validation precision@10 is measured after every epoch and the best epoch is kept. A first run chose MLP size 64 and penalty 0.1 at the edges of a smaller grid; the grid was extended, and they won again.

NeuMF was tried three ways: from a random start (learning rate 0.001), and pre-trained, as in the paper: the trained GMF and MLP joined, each with half its output weights, then fine-tuned at 0.0001 or 0.00001. The pre-trained versions were also scored at epoch 0, before any fine-tuning.

REAL-WORLD SCENARIO

Two musicians rehearsed separately and are good. Rehearsing together might help, or the first take might be the best.

ACTUAL OUTPUT

```
GMF: validation precision@10 at its best epoch (of 20)
  size  32: lam 0.003: 0.1240 (epoch 19); lam 0.01: 0.1191 (epoch 20); lam 0.03: 0.0975 (epoch 19)
  size  64: lam 0.003: 0.1240 (epoch 13); lam 0.01: 0.1270 (epoch 17); lam 0.03: 0.1063 (epoch 20)
  size 128: lam 0.003: 0.1213 (epoch 9); lam 0.01: 0.1238 (epoch 15); lam 0.03: 0.1097 (epoch 20)
  chosen GMF: size 64, lam 0.01, 17 epochs (validation 0.1270)
MLP: validation precision@10 at its best epoch (of 20)
  size  32: lam 0.03: 0.1220 (epoch 18); lam 0.1: 0.1212 (epoch 17); lam 1: 0.1210 (epoch 17)
  size  64: lam 0.03: 0.1179 (epoch 11); lam 0.1: 0.1226 (epoch 15); lam 1: 0.1217 (epoch 20)
  size 128: lam 0.03: 0.1219 (epoch 11); lam 0.1: 0.1221 (epoch 10); lam 1: 0.1192 (epoch 20)
  chosen MLP: size 64, lam 0.1, 15 epochs (validation 0.1226)
NeuMF, from scratch, lr 0.001: best validation 0.1187 at epoch 11; by epoch from 1: 0.071, 0.095, 0.098, 0.110, 0.115,
    0.118, 0.116, 0.118, 0.115, 0.115, 0.119, 0.112, 0.112, 0.111, 0.110, 0.113, 0.103, 0.105, 0.106, 0.093
NeuMF, pre-trained, lr 0.0001: best validation 0.1319 at epoch 0; by epoch from 0: 0.132, 0.132, 0.130, 0.125, 0.122, 0.120,
    0.118, 0.113, 0.112, 0.111, 0.109, 0.109, 0.107, 0.105, 0.102, 0.098, 0.103, 0.103, 0.101, 0.101, 0.098
NeuMF, pre-trained, lr 1e-05: best validation 0.1319 at epoch 0; by epoch from 0: 0.132, 0.131, 0.130, 0.130, 0.131, 0.131,
    0.131, 0.131, 0.130, 0.130, 0.130, 0.130, 0.130, 0.130, 0.129, 0.128, 0.128, 0.127, 0.127, 0.127, 0.127
for reference, Day 158's validation precision@10: implicit ALS 0.1374, PureSVD 0.1313
Actual output from this lesson's run.
```

What it means

GMF chose size 64 and penalty 0.01 (0.1270); MLP size 64 and penalty 0.1 (0.1226). The MLP grid is flat: every cell lies between 0.1179 and 0.1226. NeuMF from scratch peaked at 0.1187 and then fell, worse than either tower alone.

Pre-trained NeuMF scored 0.1319 at epoch 0, the best of any network, and fine-tuning only lowered it: to 0.098 at 0.0001, and slowly towards 0.127 even at 0.00001. So the chosen NeuMF is the untouched combination; its printed learning rate is never used, because it trains for 0 epochs. A likely reason, not tested here: both towers were already stopped at their best validation epoch, so any further training on the same data overfits. For reference, Day 158's implicit ALS scored 0.1374 on the same validation set.

WHY CHOOSE THIS?

Scoring every epoch and keeping the best (early stopping) is cheap and caught both the peak and the decline.

WHY NOT THIS? (TRADEOFFS)

A grid of 18 cells at 20 epochs took most of the 2.5-hour run; a flat grid like MLP's says the settings matter less than the model.

4. The week's final comparison on Day 154's test

DEFINITION — start here, no prior knowledge assumed

Every model is refitted on all 80,000 training ratings with its chosen settings: popularity, co-occurrence, PureSVD, implicit ALS (binary and with stars as confidence), GMF, MLP and NeuMF. Day 157's blend combines implicit ALS and NeuMF with weights fitted on validation cells, from versions trained without validation. A control shuffles NeuMF's user embeddings between users, removing everything personal while keeping the films' side.

Day 159's measures are used throughout, plus the number of distinct genres in a list (from Day 156's flags), and a paired bootstrap over users on graded NDCG@10.

REAL-WORLD SCENARIO

A new player joins a team on trial. The coach compares them with the current starters in the same match, not with last season's substitutes.

ACTUAL OUTPUT

```
models refitted on all 80000 training ratings (275 s); chosen: NeuMF, pre-trained, lr 0.0001
Day 157's blend fitted on 36950 validation cells: intercept +0.167, implicit ALS +0.190, NeuMF +0.035
                      test @10 precision  recall    NDCG  graded coverage median pop. personal. genres
                  popularity    0.1948  0.1195  0.2267  0.1698    0.030         346     0.504  10.70
              co-occurrence    0.2930  0.1948  0.3606  0.2806    0.114         247     0.844  10.63
                     PureSVD    0.3507  0.2301  0.4218  0.3250    0.233         216     0.927   9.76
                implicit ALS    0.3672  0.2438  0.4423  0.3448    0.237         233     0.916   9.98
          implicit ALS, stars    0.3703  0.2387  0.4478  0.3561    0.199         241     0.906   9.95
                         GMF    0.3311  0.2154  0.3933  0.3052    0.313         214     0.924   9.58
                         MLP    0.3209  0.2083  0.3778  0.2823    0.353         212     0.930   9.78
                       NeuMF    0.3475  0.2275  0.4149  0.3173    0.303         221     0.922   9.72
   blend: implicit ALS + NeuMF    0.3692  0.2443  0.4440  0.3461    0.235         233     0.916   9.96
 control: NeuMF, users shuffled    0.1256  0.0709  0.1451  0.1048    0.259         239     0.902   9.45
paired bootstrap over users, graded NDCG@10:
                       NeuMF - implicit ALS: -0.0275, 95% interval [-0.0357, -0.0194], resamples with no gain 100.0%
                       NeuMF - PureSVD     : -0.0077, 95% interval [-0.0169, +0.0017], resamples with no gain 94.8%
  blend: implicit ALS + NeuMF - implicit ALS: +0.0013, 95% interval [+0.0001, +0.0025], resamples with no gain 2.0%
Actual output from this lesson's run.
```

What it means

Implicit ALS stays on top. NeuMF (0.3475) beat GMF (0.3311) and MLP (0.3209), and trailed implicit ALS by -0.0275 graded NDCG, interval [-0.0357, -0.0194]. Against PureSVD the interval [-0.0169, +0.0017] includes 0: no clear difference. The control fell to 0.1256, below popularity, so the networks' accuracy does come from the users' embeddings.

The networks spread their lists more widely: coverage 0.303 to 0.353, against implicit ALS's 0.237. The blend's weight on NeuMF was small (+0.035 against +0.190) and so was its gain: +0.0013, interval [+0.0001, +0.0025], just above 0 and too small to matter. These results agree with Rendle and others (2020), who found well-tuned matrix factorisation matching or beating NCF.

WHY CHOOSE THIS?

One test, one split, every model tuned the same way on validation: the comparison is fair.

WHY NOT THIS? (TRADEOFFS)

One dataset and one split; a larger catalogue or other side information might favour the network.

5. The paper's protocol: leave-last-out, 100 sampled negatives

DEFINITION — start here, no prior knowledge assumed

The NCF paper held out each user's latest film and ranked it among 100 sampled unwatched films (Day 159's shortcut). Every model is refitted without those films and scored both ways: against every film, and against 100 sampled ones.

With one film per user, a hit rate is a share of 943 yes/no results, so its standard error is $\sqrt{(p(1 – p) / 943)}$.

REAL-WORLD SCENARIO

Ranking sprinters by one heat each: a single race says little about who is faster when the times are close.

MATHEMATICS

$$\textit{standard error of a hit rate} = \sqrt{(p(1 – p) / n)},\ n = 943 \textit{ users}$$

ACTUAL OUTPUT

```
943 held-out films, models refitted (336 s)
                     all films ranked: HR@10  NDCG@10    100 sampled: HR@10  NDCG@10
          popularity                   0.0859   0.0449                0.4093   0.2287
      co-occurrence                    0.1092   0.0563                0.5090   0.2910
             PureSVD                   0.1410   0.0708                0.6458   0.3776
        implicit ALS                   0.1474   0.0752                0.6638   0.3865
 implicit ALS, stars                   0.1432   0.0724                0.6511   0.3778
                GMF                    0.1188   0.0633                0.6564   0.3799
                MLP                    0.1485   0.0769                0.6288   0.3768
              NeuMF                    0.1453   0.0725                0.6649   0.3946
standard error of a hit rate near 0.15 over 943 users: 0.0116
standard error of a hit rate near 0.65 over 943 users: 0.0155
Actual output from this lesson's run.
```

What it means

With 100 sampled negatives NeuMF comes first (0.6649), just ahead of implicit ALS (0.6638), and GMF (0.6564) beats PureSVD. Ranked against every film the order changes: MLP leads (0.1485), implicit ALS 0.1474, NeuMF 0.1453, and GMF drops to 0.1188. Day 159 found the sampled shortcut kept the order; with the networks included, it does not.

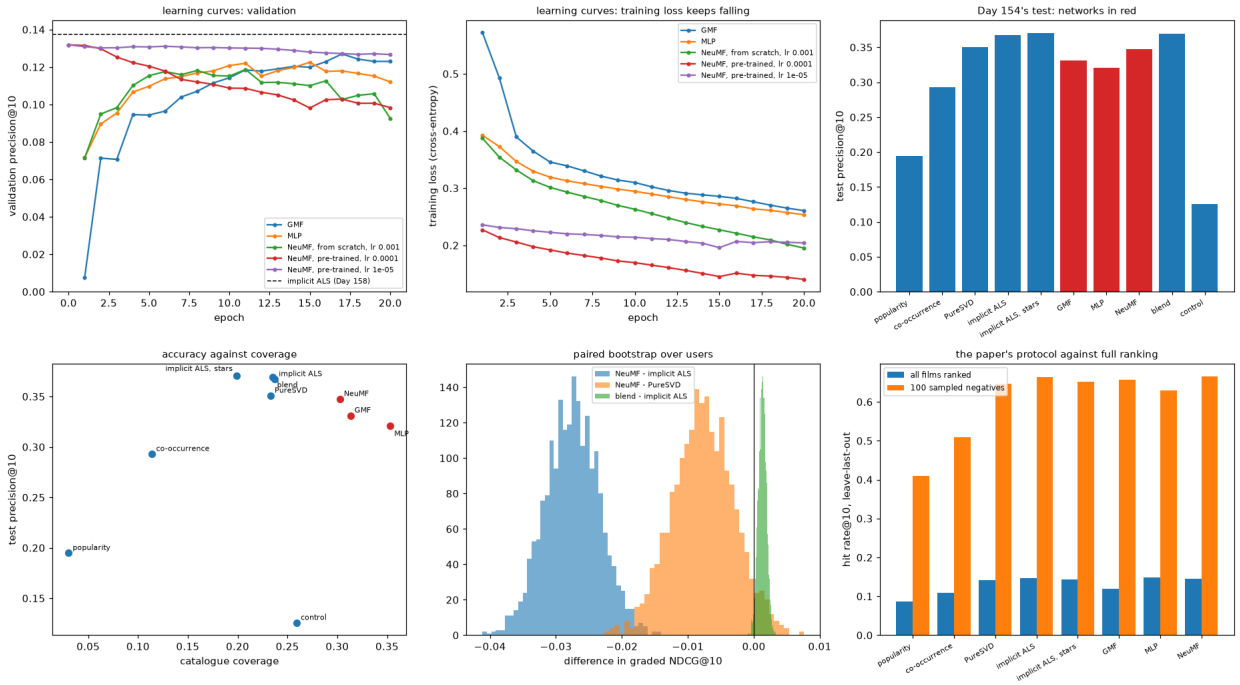
Every gap among the top models is smaller than one standard error (0.0116 near 0.15, 0.0155 near 0.65), so a single held-out film per user cannot separate them. Section 4, with about 20 test films per user, can.

WHY CHOOSE THIS?

Matches the published numbers' protocol, so a paper's claims can be checked directly.

WHY NOT THIS? (TRADEOFFS)

Small, noisy samples and a shortcut that can reorder models; Krichene and Rendle (2020) recommend ranking every item.



Top: validation learning curves (pre-trained NeuMF starts at epoch 0); training loss keeps falling while validation does not; test precision@10, networks in red. Bottom: precision against coverage; paired-bootstrap differences in graded NDCG; hit rate with every film ranked and with 100 sampled negatives.

Interview preparation — technical questions

Q: How does GMF relate to matrix factorisation?

A: GMF scores $h \cdot (p_u * q_i)$. With every h equal to 1 that is the dot product $p_u \cdot q_i$, so matrix factorisation is a special case.

Q: Why sample negatives, and how many?

A: Implicit data has only positives; the network needs 0s. The paper used 4 per watched film, redrawn each epoch, which is what is used here.

Q: Why np.add.at in the embedding gradient?

A: A user or film can appear several times in one batch; G[u] += x keeps only one copy, np.add.at adds them all.

Q: What went wrong with L2 inside Adam here?

A: Decay on every weight collapsed GMF to zero (validation 0.027). A likely reason, not tested here: Adam gives a tiny decay term full-sized steps while data gradients are near zero. Penalising only the batch's rows fixed it.

Q: How would you decide whether NCF is worth deploying?

A: Same split and tuning rules as the existing model, full-ranking measures, a paired bootstrap, and a control. Here it trailed implicit ALS by -0.0275 graded NDCG with an interval excluding 0.

Interview preparation — theoretical questions

Q: Why can a sampled evaluation crown the wrong model?

A: Ranking against 100 random films mostly tests separating the target from obscure films; full ranking tests it against popular ones too. Here NeuMF led sampled and MLP led full ranking.

Q: Why did pre-trained NeuMF not improve with fine-tuning?

A: Not tested here; a likely reason is that both towers were already stopped at their best validation epoch, so further training overfits.

Q: What does the shuffled-user control show?

A: Giving each user someone else's embeddings drops precision to 0.1256, below popularity: the model's skill is personal.

Q: Is a more expressive model always better?

A: No. A dot product is a strong, easy-to-fit bias; the network must learn it from data and has more ways to overfit.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> The week's parts made a fair test for a neural recommender: the same data, split, tuning rules and measures as every other model. NCF learned, beat its own control by a wide margin, and spread its lists more widely, but did not beat implicit ALS on full ranking, and fine-tuning NeuMF never helped. The paper's sampled protocol would have told a different story.

- Test precision@10: implicit ALS 0.3672, NeuMF 0.3475, GMF 0.3311, MLP 0.3209, control 0.1256.
- NeuMF – implicit ALS, graded NDCG: -0.0275 [-0.0357, -0.0194].
- L2 decay on every weight inside Adam collapsed GMF (0.027 flat); the per-batch penalty reached 0.088.
- Leave-last-out hit rate: NeuMF first with sampled negatives (0.6649), MLP first with every film ranked (0.1485), all within one standard error.

Suggested watch order, if starting from zero

- 1. Neural Collaborative Filtering (NCF) for recommendation systems explained (Data Science in your pocket) — the idea of NCF, GMF, the MLP and NeuMF, explained from the paper.
- 2. Neural Collaborative Filtering (NCF) Explanation & Implementation in Pytorch (Steeve Huang) — the paper walked through, then implemented in PyTorch.

Day 161 preview

- **Week 24: model evaluation and statistics, a deep dive.** Day 161 revisits the bias–variance tradeoff with real learning curves; the week goes on to hypothesis tests, bootstrapping, multiple comparisons, calibration, fairness metrics and explainability.